

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор
должность, уч. степень, звание

подпись, дата

А.В. Бржезовский
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

ЗАПРОСЫ НА ЯЗЫКЕ SQL:
ПОДЗАПРОСЫ

по курсу: Методы и средства проектирования
информационных систем и технологий

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

И.А. Лаврешин
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Научиться формулировать условия через EXISTS и NOT EXISTS и вложенные подзапросы, использовать подзапросы в командах INSERT, UPDATE и DELETE и закрепить правило проверки NULL.

2 Задание и исходные данные

Реализовать запросы ж – и, указанные в варианте задания, с использованием [not] exists и подзапросов.

Самостоятельно предложить и реализовать запросы, демонстрирующие использование подзапросов в операторах манипулирования данными (п. 5.1).

С помощью [not] exists реализовать запросы, разработанные в п. 4 для иллюстрации использования теоретико-множественных операций, показать различие в выполнении запросов с [not] exists и теоретико-множественными операциями при наличии в таблицах null-значений (п. 5.3).

3 Ход выполнения задания

ж) Подразделения без совместителей. Выводим все подразделения, для которых NOT EXISTS ни одной строки прием с совместительство = B'1'. Так корректно выражается «нигде не встречается» через вложенный подзапрос.

з) Все должности представлены в штатном расписании подразделения. Логика: для каждого подразделения не должно существовать такой должности из набора, чтобы для неё NOT EXISTS строки штатное_расписание для этого подразделения. Двойное NOT EXISTS задаёт универсальную кванторную форму («для каждой должности найдётся штатная строка»).

и) У сотрудника все дети одного пола. Требование «есть дети» — EXISTS по дети. Одновременно исключаем пары разных полов среди детей этого же сотрудника: NOT EXISTS по декартову произведению двух строк дети с условием ид1 < ид2 и пол1 <> пол2. Результат сортируется по ФИО.

На рисунках 1 – 4 представлены результаты работы программы и её листинг.

```

1  SELECT
2      пд.ид,
3      пд.наименование AS подразделение
4  FROM подразделение пд
5  WHERE NOT EXISTS (
6      SELECT 1
7      FROM прием пр
8      WHERE пр.ид_подразделения = пд.ид
9             AND пр.совместительство = B'1'
10 )
11 ORDER BY пд.ид;
12
13 SELECT
14     пд.ид,
15     пд.наименование AS подразделение
16 FROM подразделение пд
17 WHERE NOT EXISTS (
18     SELECT 1
19     FROM должность д
20     WHERE д.ид IN (1, 2, 3, 4)
21            AND NOT EXISTS (
22                SELECT 1
23                FROM штатное_расписание шр
24                WHERE шр.ид_подразделения = пд.ид
25                       AND шр.ид_должности = д.ид
26            )
27 )
28 ORDER BY пд.ид;
29
30 SELECT
31     с.ид,
32     с.фio
33 FROM сотрудник с
34 WHERE EXISTS (
35     SELECT 1
36     FROM дети д
37     WHERE д.ид_сотрудника = с.ид
38 )
39    AND NOT EXISTS (
40     SELECT 1
41     FROM дети д1
42            CROSS JOIN дети д2
43     WHERE д1.ид_сотрудника = с.ид
44            AND д2.ид_сотрудника = с.ид
45            AND д1.ид < д2.ид
46            AND д1.пол <> д2.пол
47 )
48 ORDER BY с.фio;

```

Рисунок 1 – Запросы ж–и

	ид [PK] integer	подразделение character varying (200)
1	1	ООО «Промтех»
2	2	Цех механической обработки
3	3	Отдел главного конструктора

Рисунок 2 – Результаты запроса ж

	ид [PK] integer	подразделение character varying (200)
1	3	Отдел главного конструктора

Рисунок 3 – Результаты запроса з

	ид [PK] integer	фio character varying (200)
1	2	Петрова Мария Сергеевна
2	3	Сидоров Пётр Алексеевич

Рисунок 4 – Результаты запроса и

Создаётся временная таблица `lr5_tmp_ins` с колонками “идентификатор сотрудника” и поясняющим текстом. Команда `INSERT INTO ... SELECT` заполняет её строками из таблицы `сотрудник`, но только для тех сотрудников, у которых в таблице `дети` есть хотя бы одна запись. Проверка задана через предикат `EXISTS (SELECT 1 FROM дети ...)` с корреляцией по `с.ид`: подзапрос не возвращает “данные для показа”, а лишь даёт ответ “есть / нет” подходящей строки, что соответствует типичному стилю использования `EXISTS` в условиях отбора. В поле комментария для наглядности подставляется константа 'есть дети'. Так демонстрируется `INSERT`, в котором источником строк служит подзапрос с `EXISTS`. Результаты представлены на рисунках 5 – 6.

```

1  DROP TABLE IF EXISTS lr5_tmp_ins;
2
3  CREATE TEMP TABLE lr5_tmp_ins (
4      ид_сотрудника int PRIMARY KEY,
5      комментарий text
6  );
7
8  INSERT INTO lr5_tmp_ins (ид_сотрудника, комментарий)
9  SELECT с.ид, 'есть дети'
10 FROM сотрудник с
11 WHERE EXISTS (SELECT 1 FROM дети д WHERE д.ид_сотрудника = с.ид);
12
13 SELECT * FROM lr5_tmp_ins ORDER BY ид_сотрудника;
```

Рисунок 5 – Первый запрос

	ид_сотрудника [PK] integer	комментарий text
1	1	есть дети
2	2	есть дети
3	3	есть дети
4	4	есть дети
5	6	есть дети

Рисунок 6 – Результаты первого запроса

Таблица lr5_tmp_upd изначально заполняется простым отбором: для каждого подразделения из справочника подразделение вставляется его ид, а счётчик обнуляется. Затем выполняется UPDATE: для каждой строки псевдонима и в столбец "счётчик" подставляется результат скалярного подзапроса COUNT(*) по таблице прием, ограниченного тем же подразделением. Подзапрос коррелирует с текущей строкой обновляемой таблицы: для разных строк и подсчитывается разное число приёмов. Так иллюстрируется п. 5.1 — подзапрос в списке присваивания в UPDATE. Результаты представлены на рисунках 7 – 8.

```

1  DROP TABLE IF EXISTS lr5_tmp_upd;
2
3  CREATE TEMP TABLE lr5_tmp_upd (
4      ид int PRIMARY KEY,
5      счётчик int NOT NULL DEFAULT 0
6  );
7
8  INSERT INTO lr5_tmp_upd (ид, счётчик)
9  SELECT ид, 0 FROM подразделение;
10
11 UPDATE lr5_tmp_upd u
12 SET счётчик = (
13     SELECT COUNT(*)::int
14     FROM прием пр
15     WHERE пр.ид_подразделения = u.ид
16 );
17
18 SELECT * FROM lr5_tmp_upd ORDER BY ид;

```

Рисунок 7 – Второй запрос

	ид [PK] integer	счётчик integer
1	1	0
2	2	4
3	3	7
4	4	1

Рисунок 8 – Результаты второго запроса

В lr5_tmp_del попадают идентификаторы сотрудников с идентификатором не больше трёх — это учебный "пробный" набор без претензии на смысл для предметной области. Команда DELETE FROM lr5_tmp_del удаляет из временной таблицы те строки, для которых в основной таблице увольнение существует запись с тем же ид_сотрудника. Условие

записано как EXISTS (SELECT 1 FROM увольнение ...) с привязкой к d.ид. Смысл демонстрации: из исходного списка ид "кандидатов" убираются те, кто в базе считается уволенным, в временной таблице остаются только те ид из выбранного диапазона, для кого в увольнение ничего нет — то есть примитивная имитация "удалить из выборки уже не работающих". Основные таблицы сотрудник и увольнение при этом не изменяются, меняется только содержимое временной таблицы. Результаты представлены на рисунках 9 – 10.

```
1 DROP TABLE IF EXISTS lr5_tmp_del;
2
3 CREATE TEMP TABLE lr5_tmp_del (
4     ид int PRIMARY KEY
5 );
6
7 INSERT INTO lr5_tmp_del (ид)
8 SELECT с.ид FROM сотрудник с WHERE с.ид <= 3;
9
10 DELETE FROM lr5_tmp_del d
11 WHERE EXISTS (
12     SELECT 1
13     FROM увольнение у
14     WHERE у.ид_сотрудника = d.ид
15 );
16
17 SELECT * FROM lr5_tmp_del ORDER BY ид;
```

Рисунок 9 – Третий запрос

	ид [PK] integer	
1		1
2		2
3		3

Рисунок 10 – Результаты третьего запроса

4 Вывод

EXISTS позволяет выразить “есть хотя бы один” и через отрицание — “нет ни одного”, что удобно для сложных кванторных формулировок. Подзапросы в DML локализуют правила отбора строк в самой команде модификации. Эквивалент INTERSECT/EXCEPT через EXISTS не завязан на операции множества и переносимее в СУБД без полного набора ключевых слов ALL. Корректная проверка неизвестного значения выполняется только через IS NULL / IS NOT NULL.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
-- 01_queries_gde.sql
```